



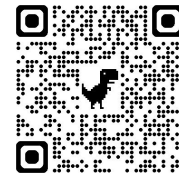
Postgres e SQL - Parte #03:

Consultando múltiplas tabelas com JOINS

Banco de Dados

Prof. Dr. Carlos Melo

 carlos.melo@upe.br



Acompanhamento do Projeto (15 min)

- Status atual do projeto
- Dificuldades encontradas
- Próximos passos



O Problema

- Na última aula, usamos o JOIN para unir a nossa fragmentação. O gerente pediu:
 - "Quero ver o nome de todos os funcionários e os seus departamentos", e nós entregamos
- A empresa cresceu e não queremos ler uma lista com 10000 nomes de funcionários e departamentos



O Problema

- A nossa meta agora é responder as seguintes perguntas:
 - Qual é o gasto total com salários na empresa?
 - Qual é a média salarial do nosso setor de TI?
 - Quantos projetos temos ativos no momento?



Funções de Agregação

- O PostgreSQL nos fornece 5 funções nativas para resumir colunas inteiras:
 1. **COUNT()**: Conta a quantidade de registros
 2. **SUM()**: Soma os valores de uma coluna numérica
 3. **AVG()**: Calcula a média aritmética
 4. **MAX()**: Retorna o maior valor (teto)
 5. **MIN()**: Retorna o menor valor (piso)
- > Com exceção do **COUNT(*)**, as demais funções de agregação ignoram campos em branco (**NULL**)



Agregando o Banco

- Se não usarmos nenhum filtro, o SQL transforma a tabela inteira em um único bloco e faz o cálculo
- Para o exemplo de folha de pagamento teríamos:

```
1 SELECT
2     COUNT(*) AS total_funcionarios,
3     SUM(salario) AS custo_total,
4     AVG(salario) AS media_salarial,
5     MAX(salario) AS maior_salario
6 FROM empregado;
```



Agrupamento

- Queremos descobrir o gasto médio por departamento
- Como implementar essa consulta?



```
1 SELECT numero_de departamento,  
   SUM(salario)  
2 FROM emprega
```



Agrupamento

- Pedimos um dado com múltiplas linhas (número de departamento) e um resumo estatístico (média)
- Ele não sabe como juntar ambas as coisas
- É preciso realizar um fatiamento (*slice*) e para isso usamos a cláusula **GROUP BY**



GROUP BY

- O **GROUP BY** agrupa linhas que possuem valores idênticos nas colunas especificadas, permitindo calcular agregações por grupo
- O banco de dados primeiro separa os registros em grupos e, só depois, aplica a função de agregação (**SUM**, **AVG**) individualmente dentro de cada grupo



```
1 SELECT numero_departamento,  
   SUM(salario) AS custo_setor  
2 FROM empregado  
3 GROUP BY numero_departamento;
```



Exercícios

- Conte o número de empregados por departamento:
 - Use **COUNT()** o parâmetro pode ser uma coluna ou a wildcard, se * contará os nulos, se especificado só os não nulos.



```
1 SELECT numero_departamento, COUNT(*)  
2 FROM empregado  
3 GROUP BY numero_departamento;
```



GROUP BY

- Agrupamos os departamentos pelo número. Mas o gerente não sabe o significado de "Departamento 1". Ele quer ver o respectivo nome do setor na tela sem a necessidade de uma nova consulta
- Precisaremos usar de um **JOIN** e ele virá antes da cláusula

GROUP BY



```
1 SELECT d.nome AS setor, SUM(e.salario) AS custo_total
2 FROM empregado e
3 INNER JOIN departamento d ON e.numero_departamento = d.numero
4 GROUP BY d.nome;
```



Filtros

- O gerente lançou a braba:
 - "Quero ver apenas os departamentos onde o custo total de salários ultrapassa R\$10.000,00"
- Como você faria isso?





HAVING

- ○ **WHERE** filtra linhas individuais antes do agrupamento.
 - Ele não enxerga o total.
- Para filtrar resultados de funções de agregação, usamos exclusivamente a cláusula **HAVING**.
- A cláusula **HAVING** entra sempre depois do **GROUP BY**



Exercício

- Calcule o custo total por setor, considerando apenas funcionários que ganham mais de 4000, e mostrando apenas setores cujo custo final passe de 10000

```
1 SELECT d.nome AS setor, SUM(e.salario) AS custo_total
2 FROM empregado e
3 INNER JOIN departamento d ON e.numero_departamento = d.numero
4 WHERE e.salario > 4000.00
5 GROUP BY d.nome
6 HAVING SUM(e.salario) > 10000.00;
```



Subconsultas (Subquery)

- Uma subconsulta é um **SELECT** dentro de outro **SELECT**
- Quem na empresa ganha mais que a média?
- Não é possível usar o **WHERE** `salario > AVG(salario)`, pois agregação não funciona no **WHERE**
- A ideia:

```
1 SELECT nome, salario
2 FROM empregado
3 WHERE salario > (SELECT AVG(salario) FROM empregado);
```





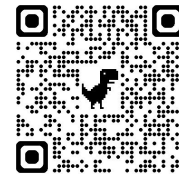
Exercícios

1. **GROUP BY**: Conte quantos funcionários trabalham em cada departamento. Exiba o número do departamento e a quantidade de pessoas (Dica: Use COUNT).
2. **JOIN + GROUP BY**: O gerente quer saber a quantidade total de horas alocadas por projeto. Exiba o nome do projeto e a soma das horas (**SUM**).



Exercícios

1. **HAVING**: A partir da consulta anterior, altere o código para mostrar apenas os projetos que possuem mais de 30 horas totais alocadas.
2. **Subquery**: Descubra e liste o nome e o salário dos funcionários que possuem o MAIOR salário de toda a empresa.



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br